FinSight AI - Multi-Agent Financial Intelligence Platform

Technical Report | Adish Padalia | May 2026

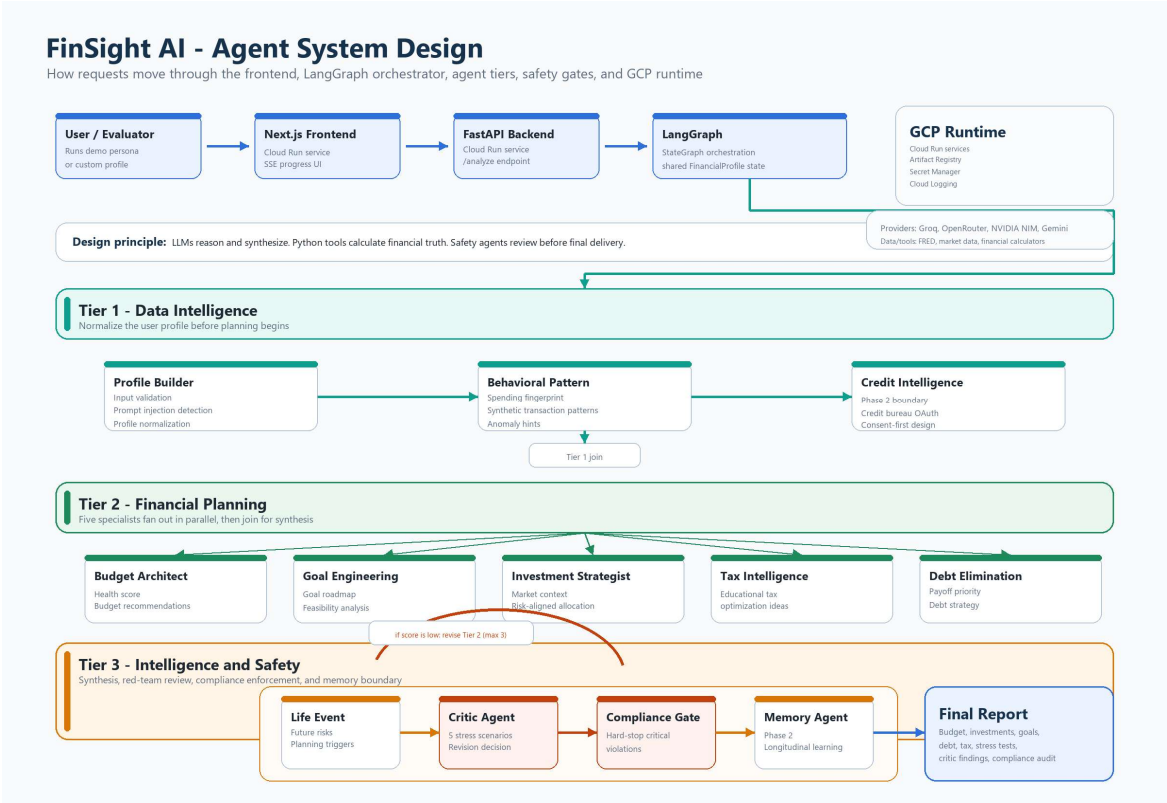
Frontend: https://finsight-frontend-ot4crz4jza-uc.a.run.app | Backend: https://finsight-api-ot4crz4jza-uc.a.run.app

Repo: https://github.com/AdishPadalia26/Finsight-AI

Project summary: FinSight AI is a live GCP-deployed multi-agent system that analyzes a user financial profile through specialized agents, deterministic financial tools, LLM reasoning, critic review, and compliance guardrails.

1. Multi-Agent Architecture

FinSight AI uses a 12-agent architecture organized into three tiers: Data Intelligence, Financial Planning, and Intelligence/Safety. The system is designed this way because financial planning is not a single-step generation task; it requires profile understanding, specialized analysis, adversarial review, and compliance control. The architecture includes 10 implemented LLM-driven agents and 2 Phase 2 architected agents, with deterministic internal nodes for stress testing and final report assembly.



Tier	Agents	Responsibility and boundary	
Tier 1: Data Intelligence	Profile Builder, Behavioral Pattern, Credit Intelligence	Profile Builder validates and normalizes raw user input; Behavioral Pattern infers spending behavior from profile/demo transaction patterns; Credit Intelligence is scoped as Phase 2 because real credit analysis requires consent-based bureau APIs.	

Tier 2: Financial Planning	Budget Architect, Goal Engineering, Investment Strategist, Tax Intelligence, Debt Elimination	Each agent owns one planning domain. Budget handles cash-flow and health scoring; goals handles feasibility; investment handles risk-aligned allocation with market context; tax provides educational opportunities; debt handles payoff strategy.	
Tier 3: Intelligence and Safety	Life Event Anticipation, Critic Agent, Compliance Gate, Memory Agent	Life Event anticipates future risks; Critic red-teams recommendations and can trigger revision; Compliance Gate reviews final output; Memory is Phase 2 for longitudinal personalization.	

Agents communicate through a LangGraph StateGraph using a shared FinancialProfile state object. Each node reads only the fields it needs and writes a bounded output back to state, which keeps responsibilities inspectable and avoids uncontrolled agent-to-agent behavior. Execution is mixed: Profile Builder runs sequentially first, Tier 1 enrichment runs in parallel, Tier 2 planning agents run in parallel, then synthesis and safety stages run sequentially. The Critic Agent introduces a controlled negotiation pattern: if stress scores are weak, the graph routes back to Tier 2 for revision, capped at three loops to prevent runaway execution.

2. Security, Safety, and Guardrails

Because the use case involves financial guidance, the system uses defense in depth rather than relying on a single prompt. Input validation happens before normal LLM execution. The Profile Builder detects prompt-injection patterns such as instruction override attempts, fake system messages, role changes, and requests to ignore previous instructions. User input is then normalized into a structured profile so downstream agents operate on typed fields rather than unrestricted text.

LLM guardrails are applied at the agent level and at the final output level. Each agent has a narrow system prompt, a specific responsibility, scoped inputs, and an expected output contract. Agents do not receive broad tool access or permission to perform external actions. Financial math such as savings rate, debt-to-income ratio, emergency fund months, health score, and stress-test values is computed deterministically in Python rather than generated by an LLM. This reduces hallucinated numbers and makes the recommendations more auditable.

The output layer includes scanning for risky financial language, unsupported guarantees, direct ticker recommendations, tax certainty, and language that could imply licensed advice. The Critic Agent stress-tests plans against adverse scenarios, while the Compliance Gate performs the final safety review and can flag, rewrite, disclaim, or hard-stop critical violations. PII and secrets are handled separately: logs use session-level metadata instead of raw sensitive profiles, provider keys are stored through Google Secret Manager, and deployment uses GCP Workload Identity Federation rather than long-lived service account key files.

3. Implementation Approach

The backend is built with Python, FastAPI, LangGraph, Pydantic-style structured state, deterministic financial utility modules, and Server-Sent Events for live pipeline visibility. The frontend is built with Next.js and exposes demo personas, agent progress, dashboards, stress-test results, explainability, and compliance output. The live deployment runs on Google Cloud Platform with Cloud Run for both backend and frontend services, Artifact Registry for container images, Secret Manager for runtime secrets, Cloud Logging for platform logs, and LangSmith for graph-level tracing.

Agents are instantiated as reusable classes and registered as nodes in the LangGraph workflow. The orchestrator controls the lifecycle: it starts at Profile Builder, fans out to parallel branches, joins results, routes through planning and safety layers, and terminates only after the Compliance Gate, final report assembly, and memory boundary. This keeps coordination centralized instead of allowing agents to call each other unpredictably.

The system handles failures through provider routing, retries, and deterministic fallbacks. LLM calls can use Groq, OpenRouter, NVIDIA NIM, and Gemini, with per-agent primary and fallback providers to reduce rate-limit fragility. If a provider fails or returns invalid output, agents either retry, fall back to another provider, or produce a deterministic fallback where safe. The graph also records pipeline errors without crashing the full user experience unless a critical safety condition requires stopping.

Evaluation focuses on correctness, safety, and end-to-end reliability. Unit tests cover financial calculators, guardrail logic, synthetic data behavior, and agent contracts. Red-team tests check prompt injection and unsafe output scenarios. End-to-end persona tests run realistic demo users through the complete pipeline, while LangSmith traces and Cloud Logging provide observability for real executions.

4. Use of AI / LLMs and Collaboration

LLMs are used where reasoning, interpretation, critique, and natural-language synthesis add value. Profile Builder uses an LLM to extract structured information from free text after injection checks. Behavioral Pattern, Budget Architect, Goal Engineering, Investment Strategist, Tax Intelligence, Debt Elimination, Life Event Anticipation, Critic, Compliance, and Personalised Advisor use LLMs for domain reasoning and explanation. LLMs are not used as the source of truth for financial calculations, compliance policy enforcement, secret handling, or logging decisions.

Agent collaboration happens through shared state and graph routing rather than informal conversation. Tier 1 agents enrich the profile; Tier 2 agents independently produce planning outputs; Tier 3 agents evaluate and constrain the combined plan. The Critic Agent is the clearest negotiation mechanism: it reviews outputs from planning agents, identifies weaknesses, and can send the workflow back for revision. The Compliance Gate then acts as a deterministic control boundary before the final response is delivered.

The key trade-off is autonomy versus control. Planning agents are given constrained autonomy to reason within their domain, but the orchestrator controls execution order, parallelism, retries, revision count, and termination. This design allows the system to benefit from LLM flexibility while keeping high-risk decisions bounded by deterministic tools, scoped prompts, output scanning, and final compliance review.

Conclusion

FinSight AI demonstrates a practical multi-agent system that delivers on its design goals through clear architecture, live deployment, specialized collaboration, and responsible LLM use. The project shows that multi-agent AI is most useful when agents have explicit boundaries, shared state is inspectable, financial truth is computed deterministically, and safety gates are part of the workflow rather than an afterthought.

Future Work

Future work for FinSight AI would focus on moving the platform from a strong proof-of-concept into a more production-ready financial intelligence system. The first priority would be completing the Phase 2 agents, especially Credit Intelligence and Memory. Credit Intelligence could integrate with consent-based credit bureau APIs to analyze utilization, payment history, score-impact factors, and debt payoff effects, while the Memory Agent could support longitudinal planning by remembering user goals, preferences, and progress across sessions with clear consent and data-retention controls. The system could also be expanded with deeper Monte Carlo goal simulations, real transaction integrations, more robust red-team testing, and broader evaluation across diverse financial personas. On the deployment side, future improvements could include richer Cloud Logging dashboards, stronger LangSmith trace analysis, automated compliance regression tests, and human-review workflows for high-risk outputs. These additions would preserve the core design principle of FinSight AI: LLMs should assist with reasoning and communication, while deterministic tools, transparent orchestration, and safety gates keep the system controlled and trustworthy.